Estimating the Number of Nodes and Paths in a DAG by Following a Random Path

Günter Rote*

October 31, 2026

Abstract

We investigate a simple method to compute an unbiased estimate of the number of nodes of a directed acyclic graph (DAG) by following a random directed path. This method is due to Leonard Pitt from 1987, and it generalizes a method of D. E. Knuth from the 1960s for estimating the size of a rooted tree. A slight variation makes it possible to estimate the number of source-sink paths.

We extend the algorithm to use many "agents" that walk cooperatively through the graph. This extension unifies and generalizes several methods that have been proposed for approximately counting the nodes of a tree.

Keywords. Approximate counting, random walk, importance sampling, polytopes

MSC2020 classification.
05C85 Graph algorithms, 05A Enumerative combinatorics, 62D Statistical sampling

Table of Contents

1 Problem statement and the Path Sampling algorithm (Algorithm P) 2
1.1 Weighted selection . . . 3
1.2 No backward proportions: counting paths . . . 4
1.3 Arbitrary arc weights . . . 4
1.4 Analysis of the expectation . . . 5
2 Experiments 6
2.1 The permutation DAG . . . 6
2.2 Cubes and Klee–Minty cubes . . . 7
3 Computing the variance of X 9
4 Importance Sampling 10
5 Employing a team of agents 11
5.1 The Team Sampling family of algorithms (TS) . . . 11
5.2 Bucket Sampling (Algorithm BS) . . . 12
5.3 Crowd Sampling (Algorithm CS) . . . 13
5.4 Correctness of the Team Sampling scheme . . . 15
5.5 Generalized Condensation . . . 17
5.6 The variance for Team Sampling strategies . . . 17
5.7 Relation to dynamic programming . . . 18

*Institut für Informatik, Freie Universität Berlin, Germany, rote@inf.fu-berlin.de, https://orcid.org/0000-0002-0351-5945

6 Extensions and potential applications **19**
6.1 Extension to arc costs . . . 19
6.2 Some applications . . . 19
6.2.1 Vertices of a polytope . . . 19
6.2.2 Triangulations of a planar point set . . . 20
6.2.3 Linear extensions of a two-dimensional partial order . . . 20
6.2.4 Pattern-avoiding permutations . . . 20
6.2.5 Polyominoes . . . 21
6.2.6 Maximal sets of an independence system . . . 21
6.3 Graphs with cycles . . . 22
7 Conclusion **22**
7.1 Choice of forward and backward proportions . . . 22
7.2 Provable approximation quality? . . . 22
A Importance Sampling for Klee–Minty cubes **25**
A.1 Importance Sampling by the size of the reachable set . . . 26
B Details about implementing Klee–Minty cubes **26**
C Purdom's partial backtracking method **28**
D Index of important notation **28**

1 Problem statement and the Path Sampling algorithm (Algorithm P)

We are given a directed acyclic graph (DAG) $G = (V, E)$ and a *cost function* $c\colon V \to \mathbb{R}$ on the nodes. The task is to estimate the total cost $c(G) := \sum_{v \in V} c(v)$.

The intended setting of the problem is that G is too large to be fully scanned, but we have access to the neighbors of a node and can thereby explore parts of G. In particular, for the basic Path Sampling algorithm, we make the following assumptions:

- There is a single *source* node s_0 from which all other nodes can be reached by directed paths.
- For any node, we can pick a *successor* node (i. e., a neighbor via an outgoing arc) uniformly at random.
- For any node v, we can determine its outdegree $d^+(v)$, i. e., the number of its *successors*, its indegree $d^-(v)$, i. e., the number of its *predecessors*, and we can access its cost $c(v)$.

The following algorithm, due to Leonard Pitt [30], computes an unbiased estimate X of $c(G)$ by walking along a directed path from the source. Figure 1 illustrates the algorithm on a small example.

Algorithm P: Path Sampling

```
    // Initialization:
    X := 0   // accumulated estimate
    u := s_0   // current node, starts at the source
    W := 1   // weight
    loop
        // visit u:
        X := X + W · c(u)   // augment the estimate
        if u has no successor, terminate and return X as the estimate
        // otherwise, proceed to a random successor v:
(R)     pick a node v among the d⁺(u) successors of u uniformly at random
(U)     W := W × d⁺(u) / d⁻(v)   // update the weight
        u := v   // go to v
```

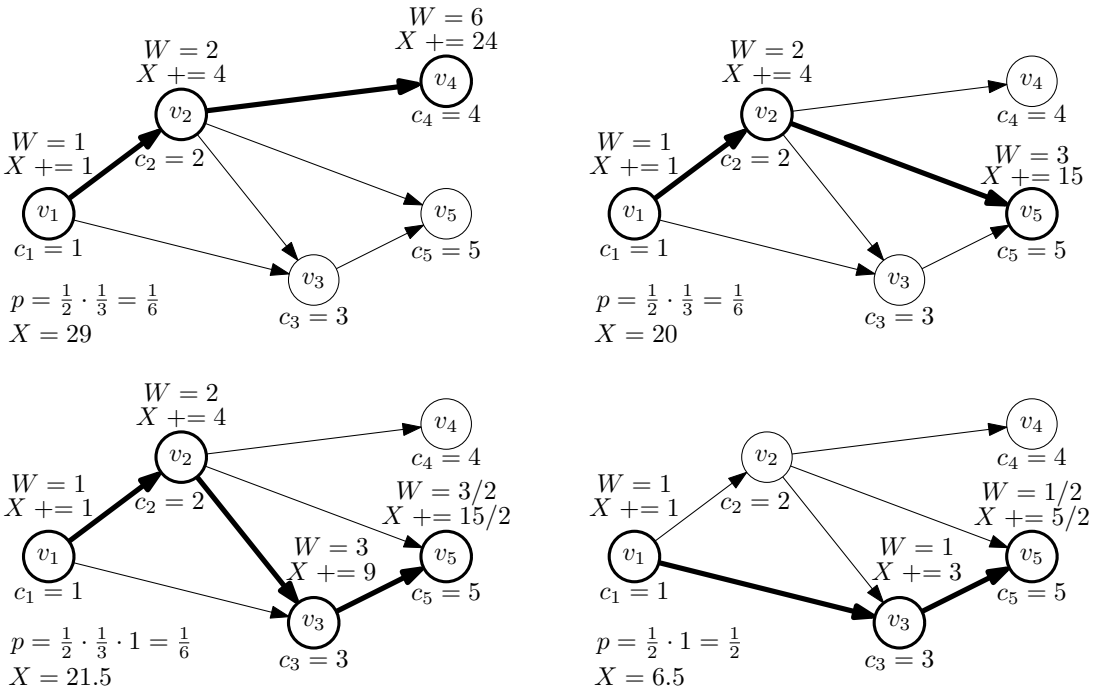


Figure 1: A DAG G with five nodes $v_1, \ldots, v_5$. The costs are $c_i = c(v_i) = i$, for a total cost of $c(G) = 1 + 2 + 3 + 4 + 5 = 15$. There are four sample paths that Algorithm P can take. The expected estimate is $E[X] = \frac{1}{6} \times 29 + \frac{1}{6} \times 20 + \frac{1}{6} \times 21.5 + \frac{1}{2} \times 6.5 = \frac{29+20+21.5+19.5}{6} = \frac{90}{6} = 15$.

Path sampling for trees. Algorithm P was originally developed for rooted trees by D. E. Knuth in 1962. It was first mentioned in a paper of Hall and Knuth [17] in 1965, and it has later been investigated in its own right [22], see also [23, Sect. 7.2.2, pp. 46–51, Corollary E]. For a tree, the indegree $d^-(v)$ is always 1, and the weight W is just the product of the outdegrees of the visited nodes.

Algorithm P is a straightforward extension to DAGs. To compensate for the fact that there can be several ways to reach a given node v, we divide W by the indegree $d^-(v)$ when going to v.

Node costs. The original purpose and the most wide-spread application of Algorithm P is to estimate the cost of backtracking or branch-and-bound algorithms. The backtracking tree or branch-and-bound tree that such an algorithm traverses may be enormous, while at other times, it is of moderate size and the algorithm terminates reasonably fast. If such a program has been running for a while, it would be useful to know whether it is just about to finish or it would continue running for years. Therefore it is desirable to have a good prediction of the remaining running time. In these applications the cost $c(v)$ is the time to process node v.

In other applications, the nodes represent some combinatorial objects, and one wishes to determine the number of these objects. In these situations, one uses uniform costs $c(v) = 1$ for all nodes v. For example, Avis and Devroye used the tree version of Algorithm P to estimate the number of vertices of a polytope [3].

If one wishes to count only the nodes with certain characteristics, this can be arranged by choosing $c(v) \in \{0, 1\}$. With appropriate costs, one can gather statistics about various properties of these objects, and of course, one can easily track several such estimates simultaneously.

1.1 Weighted selection

In line (R), the random successor does not have to be selected uniformly, but we can choose arbitrary positive transition probabilities $\vec{p}_{uv}$ subject to the constraint that they form a probability

distribution:

$$\sum_v \vec{p}_{uv} = 1, \text{ for all } u \text{ with } d^+(u) > 0 \tag{1}$$

We call the quantities $\vec{p}_{uv}$ the *forward proportions*. (The arrow above the p does not signify a vector, but it reminds us of the direction in which $\vec{p}_{uv}$ operates.)

Similarly, the incoming arcs at a node v need not be weighted equally, but they can be weighted by quantities $\overleftarrow{q}_{uv}$ that sum to 1 for the incoming arcs of each node v:

$$\sum_u \overleftarrow{q}_{uv} = 1, \text{ for all } v \neq s_0 \tag{2}$$

We call the quantities $\overleftarrow{q}_{uv}$ the *backward proportions*. (They need not have a probabilistic interpretation, and they can even be negative.)

The update of the weight when traversing the arc uv in line (U) is replaced by

$$(\mathrm{U}_q) \qquad W := W \times \overleftarrow{q}_{uv} \,/\, \vec{p}_{uv}.$$

The standard choices are $\vec{p}_{uv} = 1/d^+(u)$ and $\overleftarrow{q}_{uv} = 1/d^-(v)$.

Theorem 1. *For arbitrary positive forward proportions $\vec{p}_{uv}$ satisfying* (1) *and backward proportions $\overleftarrow{q}_{uv}$ satisfying* (2)*, Algorithm P produces an unbiased estimate of $c(G)$, i. e.,*

$$E[X] = c(G).$$

We will prove this below as a consequence of a more general statement.

1.2 No backward proportions: counting paths

If we omit the backward proportions, we get an estimate of the *number of paths starting from the source*. The weight update step (U) is replaced by

$$(\mathrm{U}_1) \qquad W := W \,/\, \vec{p}_{uv}.$$

Theorem 2. *For arbitrary positive forward proportions $\vec{p}_{uv}$ satisfying* (1)*, Algorithm P with update step* (U_1) *produces an estimate X whose expected value is*

$$E[X] = \sum_{v \in V} c(v) \cdot \#(\textit{paths from } s_0 \textit{ to } v \textit{ in } G).$$

As a special case, we can set $c(v) = 1$ if v is a sink and $c(v) = 0$ otherwise, and we get the number of source-sink paths.

1.3 Arbitrary arc weights

More generally, we can replace the backward proportions by arbitrary real arc weights $e_{uv} \in \mathbb{R}$. The weight function is extended from arcs to paths in the usual way: We define $e(P)$ as the product of the arc weights along the path P:

$$e(P) = e(u_0 u_1 \dots u_k) = e_{u_0 u_1} e_{u_1 u_2} \dots e_{u_{k-1} u_k}$$

Theorem 3. *For arbitrary positive forward proportions $\vec{p}_{uv}$ satisfying* (1) *and arbitrary arc weights e_{uv}, Algorithm P with update step*

$$(\mathrm{U}_e) \qquad W := W \times e_{uv} \,/\, \vec{p}_{uv}$$

produces an estimate X whose expected value is the quantity $c_e(G)$ defined as follows:

$$E[X] = c_e(G) := \sum_{v \in V} \sum_{\text{paths } P \text{ from } s_0 \text{ to } v} e(P) c(v)$$

1.4 Analysis of the expectation

Proof of Theorem 3. We define for every node v the random variable

$$W_v := \begin{cases} \text{the value of } W \text{ at the time that } v \text{ was visited}, & \text{if } v \text{ was visited} \\ 0, & \text{if } v \text{ was not visited} \end{cases}$$

Then $X = \sum_{v \in V} W_v \cdot c(v)$, and therefore, $E[X] = \sum_{v \in V} E[W_v] \cdot c(v)$, by linearity of expectation. The theorem is a direct consequence of the following lemma.

Lemma 4. *For every node v, $E[W_v] = \sum_{\text{paths } P \text{ from } s_0 \text{ to } v} e(P)$.*

Proof. By definition, the expectation of W_v can be written as follows:

$$E[W_v] = \sum_{\text{paths } P \text{ from } s_0 \text{ to } v} \Pr[\text{path } P \text{ is taken}] \times (W_v \text{ as a result of path } P) \tag{3}$$

For a path $P = u_0 u_1 \dots u_k$,

$$\Pr[\text{path } P \text{ is taken}] = \vec{p}_{u_0 u_1} \vec{p}_{u_1 u_2} \cdots \vec{p}_{u_{k-1} u_k}.$$

If the path P is taken, then

$$W_v = \frac{e_{u_0 u_1}}{\vec{p}_{u_0 u_1}} \cdot \frac{e_{u_1 u_2}}{\vec{p}_{u_1 u_2}} \cdots \frac{e_{u_{k-1} u_k}}{\vec{p}_{u_{k-1} u_k}}.$$

Thus, in each product term in (3), the $\vec{p}$'s cancel, and the claim of the lemma follows.

The proof of Theorem 3 is thus also complete. □

Theorem 2 is just a special case of Theorem 3 with uniform arc weights $e \equiv 1$.

Proof of Theorem 1 from Theorem 3. Here the weights e_{uw} are the backward proportions $\overleftarrow{q}_{uw}$. We only need to show the following statement:

Lemma 5. *Suppose the weights of the arcs uw are chosen as $e_{uw} = \overleftarrow{q}_{uw}$, subject to* (2)*. Then, for every node v,*

$$\sum_{\text{paths } P \text{ from } s_0 \text{ to } v} e(P) = 1.$$

Proof. This is easy to prove by induction on the length of the longest path from s_0 to v.

For the source $v = s_0$, the claim follows directly from the fact that the degenerate path $P = s_0$ with no arcs has weight 1.

For a node $v \neq s_0$, let $u_1, \dots, u_d$ be the predecessor nodes of v, with $d = d^-(v)$. We obtain

$$\begin{aligned} \sum_{\text{paths } P \text{ from } s_0 \text{ to } v} e(P) &= \sum_{i=1}^{d} \left(\sum_{\text{paths } P' \text{ from } s_0 \text{ to } u_i} e(P') \overleftarrow{q}_{u_i v} \right) \\ &= \sum_{i=1}^{d} \left(\sum_{\text{paths } P' \text{ from } s_0 \text{ to } u_i} e(P') \right) \overleftarrow{q}_{u_i v} = \sum_{i=1}^{d} \overleftarrow{q}_{u_i v} = 1, \end{aligned}$$

since each parenthesized expression in the last line is 1, by the induction hypothesis, and the backward proportions sum to 1, by assumption (2). □

If the backward proportions $\overleftarrow{q}_{uw}$ are nonnegative, there is an easy alternate proof, following Pitt [30]: We interpret $\overleftarrow{q}_{uw}$ as the transition probability of going from w to u in a random path in the reverse DAG. Then the expression $e(P)$ is the probability of traversing the path P in reverse when starting from node v. The sum $\sum e(P)$ in the lemma is the probability of arriving at the source s_0, which is of course 1.

2 Experiments

To test the accuracy of the estimation, we tried the algorithm on two families of DAGs whose size we know. Our goal was not a systematic computational study, but to get a first impression of the behavior of the basic method and its variations. The reader who is more interested in the theoretical properties of the estimate is encouraged to skip the experimental sections with the tables. The PYTHON programs by which these preliminary experiments were conducted are available in a public repository[1], together with tables of the results of experiments over a wider range of parameters.

In our experiments, we have only estimated the number of nodes (not paths), with the standard backward proportions $\tilde{q}_{uv} = 1/d^-(v)$ and uniform costs $c(v) = 1$.

Some more interesting potential applications of the method are mentioned in Section 6.2.

2.1 The permutation DAG

The nodes of the *permutation DAG* are the $n!$ permutations of the elements $\{1, 2, \ldots, n\}$. The source node is the decreasing permutation $n, n-1, \ldots, 3, 2, 1$, and the successors of a permutation are the permutations that can be reached by swapping two adjacent elements that are out of order, as in bubblesort. The sink is the sorted permutation $123\ldots n$. Figure 2 shows a random path from the source 54321 to the sink 12345 for $n = 5$, leading to the estimate $X \approx 25.6296$. (The true size is $5! = 120$.)

54321	$d^+ = 4,\ d^- = 0,\ \ W = 1,\ X = 1$
54231	$d^+ = 3,\ d^- = 1,\ \ W = 4,\ X = 5$
45231	$d^+ = 2,\ d^- = 2,\ \ W = 6,\ X = 11$
45213	$d^+ = 2,\ d^- = 2,\ \ W = 6,\ X = 17$
45123	$d^+ = 1,\ d^- = 3,\ \ W = 4,\ X = 21$
41523	$d^+ = 2,\ d^- = 2,\ \ W = 2,\ X = 23$
14523	$d^+ = 1,\ d^- = 3,\ \ W = 1.3333,\ X = 24.3333$
14253	$d^+ = 2,\ d^- = 2,\ \ W = 0.6667,\ X = 25$
12453	$d^+ = 1,\ d^- = 3,\ \ W = 0.4444,\ X = 25.4444$
12435	$d^+ = 1,\ d^- = 3,\ \ W = 0.1481,\ X = 25.5926$
12345	$d^+ = 0,\ d^- = 4,\ \ W = 0.0370,\ X = 25.6296$

Figure 2: A path in the permutation DAG

We have tested the algorithm for the permutation DAG for $n = 8$ and $n = 9$. Table 1 displays the average of up to $m = 10^7$ repetitions of Algorithm P. The true value $n!$ is displayed in the last row. We see that the values gradually converge to an estimate that is in the right ballpark.

We also show the empirical standard deviation (the square root of the empirical variance) of the first m samples, as well as the value of the largest sample. The last row gives the precise theoretical value of the standard deviation of the random variable X, computed by the method of Section 3.

We see that the standard deviation is rather high, at least one order of magnitude higher than the mean. We also see that the largest estimate is very much larger than the mean. After $m = 1000$ estimates for $n = 8$, we have seen an estimate of around 4.83×10^7, leading to the outlier mean of 77,136. These phenomena indicate that the estimate is unreliable. Indeed, in the

[1] `https://github.com/guenterrote/estimate-DAG`

context of backtracking trees, it has often been observed that, while the estimate produced by Path Sampling is unbiased, it has a high uncertainty, and the information it gives is sometimes useless.

For such situations, Chatterjee and Diaconis [9, Section 2] proposed to look at the quantity

$$\hat{\chi} = \frac{\max\{X_1, \ldots, X_m\}}{X_1 + \cdots + X_m},$$

see also [23, Eq. (34), p. 50]. A value of $\hat{\chi}$ close to 1 indicates that the mean is essentially determined by a single sample that dominates the other samples by far. Chatterjee and Diaconis argue that a reasonable policy is to trust the experiments when $\hat{\chi}$ is small. Our tables show the Chatterjee–Diaconis score $\hat{\chi}$ in the last column.

Permutations						
	$n = 8$			$n = 9$		
m	mean ± std. dev.	max.	$\hat{\chi}$	mean ± std. dev.	max.	$\hat{\chi}$
10^1	$39{,}331.8 \pm 59{,}061.9$	1.42×10^5	0.36044	$48{,}168.7 \pm 56{,}348.3$	1.68×10^5	0.34803
10^2	$28{,}772.5 \pm 64{,}855.0$	3.42×10^5	0.11875	$134{,}918.1 \pm 4.87\times10^5$	4.21×10^6	0.31195
10^3	$77{,}136.0 \pm 1.55\times10^6$	4.83×10^7	0.62640	$188{,}542.8 \pm 1.33\times10^6$	2.93×10^7	0.15514
10^4	$44{,}629.3 \pm 6.57\times10^5$	4.83×10^7	0.10826	$221{,}679.5 \pm 2.23\times10^6$	1.49×10^8	0.06720
10^5	$42{,}382.8 \pm 1.13\times10^6$	2.89×10^8	0.06821	$372{,}754.8 \pm 1.75\times10^7$	4.94×10^9	0.13250
10^6	$39{,}837.5 \pm 8.32\times10^5$	3.63×10^8	0.00912	$347{,}645.5 \pm 1.23\times10^7$	4.98×10^9	0.01433
10^7	$40{,}820.6 \pm 1.92\times10^6$	5.16×10^9	0.01265	$351{,}350.7 \pm 2.59\times10^7$	4.64×10^{10}	0.01320
	$40{,}320 \pm 1.642\times10^6$	#visited = 29		$362{,}880 \pm 1.614\times10^8$	#visited = 37	

Table 1: Algorithm P for the permutation DAG. The number of visited nodes is always $1 + \binom{n}{2}$, equaling the number of layers, because all source-sink paths have the same length.

2.2 Cubes and Klee–Minty cubes

We take the graph (i. e., the edge skeleton) of the unit cube in n dimensions and orient its edges by some generic objective function. It turns out that, on these instances, Algorithm P works impeccably! At distance j from the source, all nodes have indegree j and outdegree $n - j$. Hence the accumulated contributions to X are $1 + n + \binom{n}{2} + \binom{n}{3} + \cdots + \binom{n}{j} + \cdots = 2^n$. We always get the correct answer.

Since the behavior of Algorithm P for standard cubes is so uninteresting, we apply the algorithm to *Klee–Minty cubes* instead. Klee and Minty, in 1972, defined a perturbation of the unit n-cube for which they could show that the simplex algorithm, with certain pivot rules, visits all 2^n vertices and thus takes exponential time [21].

Figure 3 shows a version of this polytope with integral vertices, inscribed in a box with exponentially increasing side lengths rather than a cube. The arcs are oriented in the direction of the last coordinate x_n. This orientation of the arcs agrees with the original Klee–Minty cubes. Our version of the Klee–Minty cube in n dimensions is defined by the following linear program:

$$\text{Maximize } x_n$$

subject to the following $2n$ constraints:

$$\begin{aligned} 0 \le x_1 &\le 1 \\ x_{i-1} \le x_i &\le 2^i - 1 - x_{i-1}, \quad \text{for } i = 2, \ldots, n \end{aligned} \tag{4}$$

We see that the n-dimensional Klee–Minty "box" has a *floor* facet and a *ceiling* facet, where x_n is at its lower bound $x_n = x_{n-1}$ or at its upper bound $x_n = 2^n - 1 - x_{n-1}$, respectively. Both the

floor and the ceiling are affine images of an $(n-1)$-dimensional Klee–Minty box. The floor has the original orientations, and in the ceiling, all orientations are reversed.[2]

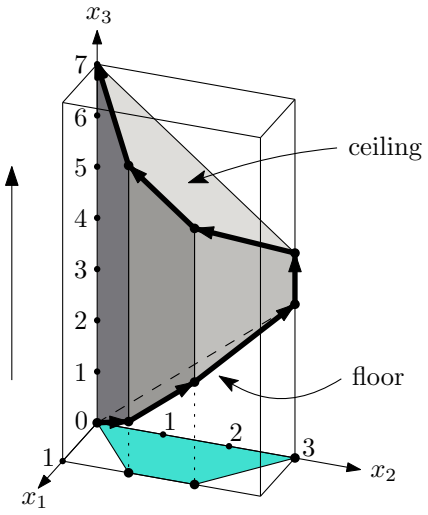


Figure 3: An integral Klee–Minty cube in $n = 3$ dimensions with the directed path through all 2^n vertices. The edges are oriented upwards in the vertical x_3-direction.

Klee–Minty cubes						
	$n = 10$			$n = 12$		
m	mean ± std. dev.	max.	$\hat{\chi}$	mean ± std. dev.	max.	$\hat{\chi}$
10^1	$795.5 \pm 2{,}170.0$	6.96×10^3	0.87488	198.8 ± 335.8	1.05×10^3	0.53028
10^2	277.6 ± 937.2	6.96×10^3	0.25073	$771.3 \pm 3{,}045.2$	2.41×10^4	0.31309
10^3	$906.3 \pm 11{,}238.4$	2.85×10^5	0.31473	$1{,}068.5 \pm 10{,}497.3$	2.81×10^5	0.26334
10^4	$485.6 \pm 4{,}998.2$	2.85×10^5	0.05874	$1{,}902.3 \pm 85{,}597.0$	8.39×10^6	0.44087
10^5	$955.3 \pm 90{,}694.9$	2.81×10^7	0.29453	$1{,}786.1 \pm 60{,}458.6$	9.70×10^6	0.05430
10^6	$795.1 \pm 59{,}515.4$	4.42×10^7	0.05565	$2{,}206.7 \pm 2.13\times10^5$	1.52×10^8	0.06900
10^7	$909.0 \pm 1.78\times10^5$	4.89×10^8	0.05384	$3{,}103.9 \pm 2.13\times10^6$	6.58×10^9	0.21208
	$1{,}024 \pm 3.246\times10^9$			$4{,}096 \pm 1.034\times10^{19}$		
	#visited: avg = 15.7, max = 78			#visited: avg = 20.6, max = 100		

Table 2: Algorithm P for Klee–Minty cubes

Algorithm P essentially performs the simplex algorithm with the *random-edge* pivot rule. For the expected number of steps of this pivot rule on Klee–Minty cubes, there is an easy upper bound of $O(n^2)$ [16]. A matching lower bound of $\Omega(n^2)$ is known only for a random start vertex [4].

Table 2 shows the results of two runs, with $n = 10$ and $n = 12$ dimensions. In comparison to the permutation DAG, we see that the estimate is less accurate, even though the cubes have far fewer nodes. The theoretical standard deviation is very large (around 10^{19} for $n = 12$), but this high standard deviation is not observed empirically. The extreme clash between the predicted and the observed standard deviation raises the suspicion of an error in the formulas or a programming error. However, for small values of n, the empirical standard deviation converges very well to the predicted standard deviation. The phenomenon is inherent in the data. The high value of the standard deviation, several orders of magnitude larger than the mean, indicates that there is an unusual situation.

[2]Our model of the Klee–Minty cube is a slight variation of the integral model proposed by Kitahara and Mizuno [20], where the floor facets lie in the coordinate hyperplanes. The coordinates x_i in (4) are related to the coordinates u_i in their model by the unimodular transformation $x_i = u_1 + \cdots + u_i$.

Appendix B gives some details about the representation that we used for the vertices of the Klee–Minty cubes.

3 Computing the variance of X

For test DAGs that are small enough so that one can iterate through all nodes, it is possible to compute the variance of the random estimate X for Algorithm P exactly.

We use the following basic result about a mixture of distributions:

Lemma 6 (Mixture of distributions)**.** *Consider d random variables $Z_1, \ldots, Z_d$ with means μ_i and variances var_i. Let Z be the mixture of these variables with proportions $\alpha_1, \ldots, \alpha_d$, that is, Z is set to Z_i with probability α_i, where $\sum_{i=1}^d \alpha_i = 1$. Then Z has mean*

$$\mu = \sum_{i=1}^{d} \alpha_i \mu_i$$

and variance

$$\mathrm{var} = \sum_{i=1}^{d} \alpha_i \big(\mathrm{var}_i + (\mu_i - \mu)^2\big) = \sum_{i=1}^{d} \alpha_i \mathrm{var}_i + \sum_{1 \le i < j \le d} \alpha_i \alpha_j (\mu_i - \mu_j)^2. \tag{5}$$

These equations can be seen as instances of the *law of total expectation* $E[Z] = E[E[Z|Y]]$ and the *law of total variance* $\mathrm{var}(Z) = E[\mathrm{var}(Z|Y)] + \mathrm{var}(E[Z|Y])$, where $Y \in \{1, \ldots, d\}$ is a random variable that determines which Z_i is chosen. The last expression in (5) is an alternative formula, which is sometimes more convenient for hand calculation when d is small.

Let us apply this to Algorithm P. For each node u, we define the random variable $D_u(W)$ to be the value of all future additions to X, assuming that u is entered with weight W. Since the operations on W consist of multiplications by random quantities that are independent of W, it is clear that $D_u(W)$ is a linear function of W, i. e.,

$$D_u(W) = F_u W,$$

for some random factor F_u that depends only on the graph G, the costs $c(v)$, the forward proportions $\vec{p}_{uv}$, and the arc weights.

We now show how the mean and the variance of the variables F_u can be computed in reverse direction of the arcs, proceeding from the end of the DAG towards the source. Eventually we arrive at the final estimate $X = F_{s_0}$, and $\mathrm{var}(F_{s_0})$ is the quantity that we are looking for.

Let u be a node with d successors $v_1, \ldots, v_d$. Then F_u is $c(u)$ plus a mixture of the d random variables $e_{uv_j}/\vec{p}_{uv_j} \cdot F_{v_j}$, each taken with probability $\alpha_j = \vec{p}_{uv_j}$. Let μ_j and var_j denote the mean and variance of F_{v_j}. Then, by Lemma 6, the mixture has mean

$$\mu_0 = \sum_{j=1}^{d} \vec{p}_{uv_j} \cdot \frac{e_{uv_j}}{\vec{p}_{uv_j}} \cdot \mu_j = \sum_{j=1}^{d} e_{uv_j} \mu_j$$

and variance

$$\begin{aligned}
\mathrm{var}(F_u) &= \sum_{j=1}^{d} \vec{p}_{uv_j} \cdot \left[\left(\frac{e_{uv_j}}{\vec{p}_{uv_j}} \right)^2 \cdot \mathrm{var}_j + \left(\frac{e_{uv_j}}{\vec{p}_{uv_j}} \mu_j - \mu_0 \right)^2 \right] \\
&= \sum_{j=1}^{d} \frac{1}{\vec{p}_{uv_j}} \left[e_{uv_j}^2 \mathrm{var}_j + \big(e_{uv_j} \mu_j - \vec{p}_{uv_j} \mu_0 \big)^2 \right]
\end{aligned}$$

To get the expectation of F_u, we must not forget to add the cost $c(u)$:

$$E[F_u] = c(u) + \mu_0 = c(u) + \sum_{j=1}^{d} e_{uv_j}\mu_j \tag{6}$$

For a node u with no successors, the formulas specialize to $\mathrm{var}(F_u) = 0$ and $E[F_u] = c(u)$.

For the special case of uniform forward proportions $\vec{p}_{uv_j} = 1/d$, we get the following formulas:

$$\mathrm{var}(F_u) = d \cdot \sum_{j=1}^{d} \left[e_{uv_j}^2 \mathrm{var}_j + \left(e_{uv_j}\mu_j - \frac{\mu_0}{d} \right)^2 \right] = d \cdot \sum_{j=1}^{d} e_{uv_j}^2 \mathrm{var}_j + \sum_{1 \le i < j \le d} \left(e_{uv_i}\mu_i - e_{uv_j}\mu_j \right)^2$$

4 Importance Sampling

How can we use the freedom to pick the successor according to an arbitrary distribution in order to get a better estimate? For a rooted tree, it is known what the ideal forward proportions $\vec{p}_{uv}$ are: They should be proportional to the cost of the subtrees rooted at the children v of node u. If the forward proportions are chosen in this way, the estimate X will always deliver the correct answer.

In practice, one can choose the probability $\vec{p}_{uv}$ of going from u to v based on the properties of v in the graph, e.g., the number of successors or predecessors, or on characteristics of the combinatorial objects that the node v represents. We are entering the realm of heuristic arguments, and the success of any particular rule depends on the type of problem.

We have tried the simple rule to choose the probabilities proportional to the outdegrees of the successor nodes v. (If the outdegree is 0, we have taken 1 instead, to avoid zero probabilities.) Nodes with large outdegree are more likely to reach many nodes. For the permutation DAG, the outdegree (and hence indegree) of a successor v differs by at most ± 1 from u, and hence the effect of the outdegree tends to amplify. This is why it seems to make sense to choose the outdegree as a criterion.

Permutations						
	$n = 8$			$n = 9$		
m	mean $\pm$ std. dev.	max.	$\hat{\chi}$	mean $\pm$ std. dev.	max.	$\hat{\chi}$
10^1	$10{,}564.8 \pm 11{,}944.9$	3.32×10^4	0.31394	$234{,}362.6 \pm 4.41\times10^5$	1.28×10^6	0.54434
10^2	$15{,}677.9 \pm 28{,}749.5$	1.85×10^5	0.11808	$315{,}348.3 \pm 1.03\times10^6$	6.34×10^6	0.20111
10^3	$47{,}912.4 \pm 4.33\times10^5$	1.23×10^7	0.25653	$281{,}586.0 \pm 1.10\times10^6$	1.40×10^7	0.04980
10^4	$39{,}265.0 \pm 2.24\times10^5$	1.23×10^7	0.03130	$312{,}165.1 \pm 3.48\times10^6$	2.36×10^8	0.07569
10^5	$40{,}970.8 \pm 3.82\times10^5$	6.63×10^7	0.01619	$411{,}450.9 \pm 2.05\times10^7$	6.05×10^9	0.14698
10^6	$39{,}777.6 \pm 4.31\times10^5$	2.35×10^8	0.00592	$360{,}495.0 \pm 1.13\times10^7$	6.05×10^9	0.01678
10^7	$40{,}289.8 \pm 5.41\times10^5$	7.25×10^8	0.00180	$372{,}028.6 \pm 4.02\times10^7$	1.01×10^{11}	0.02708
	$40{,}320 \pm 6.316\times10^5$	#visited = 29		$362{,}880 \pm 4.665\times10^7$	#visited = 37	

Table 3: Algorithm P for permutations with Importance Sampling by outdegree

The results are shown in Table 3. We see that the theoretical standard deviation goes down by a factor between 2.6 and 3.5 in comparison with Table 1. In terms of empirical behavior, the comparison is not clear. The experiments are haunted by the occasional occurrences of rare events, such as unusually early appearances of high estimates like 6.05×10^9 and 1.01×10^{11} for $n = 9$.

We tried the same method also for Klee–Minty cubes, with similarly inconclusive results. Appendix A reports these results, as well as some further experiments of only theoretical value, where the size of the reachable node set is taken into account for the forward proportions.

In another setting, namely the enumeration tree for pseudoline arrangements, it has been observed that the degrees of nodes and their children are strongly correlated, see [32, §9, Fig. 14], suggesting that importance sampling should be helpful. It indicates that forward proportions chosen proportional to the outdegrees would underestimate the true size of the subtrees, and thus the influence of the outdegrees on the probabilities should be even stronger than just proportional. However, an experimental evaluation has not been done yet. A node in some deeper level of these enumeration trees has typically several hundred to thousands of successors.

5 Employing a team of agents

5.1 The Team Sampling family of algorithms (TS)

Instead of making many independent repetitions of following a single path with Algorithm P, it is better to let several "tokens" or "cooperative agents" move through the graph simultaneously. *In this section, we assume that the arc weights e_{uv} are positive.*

We propose the following general scheme. The algorithm maintains a list A of *active nodes*, and for each element $v \in A$, a positive weight $W[v]$.

Algorithm TS: Team Sampling

1. [Initialize] $A := \{s_0\}$; $W[s_0] := 1$; $X := 0$.

2. While A is nonempty, perform either of the following two steps:

2E. [Expand successors] Choose an arbitrary node u of A, and remove it from A.
Set $X := X + W[u] \cdot c(u)$. // Augment X
For each successor v of u:
If v is not present in A, insert v into A and clear its weight: $W[v] := 0$.
Then, increase its weight to $W[v] := W[v] + W[u]e_{uv}$.

2C. [Condense] Choose an arbitrary subset S of A with at least 2 elements.
Pick a random node u from S, with probabilities proportional to the weights $W[u]$.
Replace S in A by the single element u, and set $W[u]$ to the sum of the weights in S.

3. [Terminate] In the end, return X.

The choice between Step 2E and Step 2C, and the choice of u in Step 2E or of S in Step 2C can be done according to some rule, depending on any information that has been stored or that is currently at hand, or even randomly. The node weights can be taken into account. We prove in Section 5.4 that, whatever choices the algorithm makes, it returns an unbiased estimate X of the quantity $c_e(G)$ that was defined in Theorem 3.

Algorithm P can be seen as a special case of this algorithm where Step 2E is always directly followed by Step 2C for the whole set A, i. e., the successors of the current node. (To make this correspondence precise, the forward proportions $\vec{p}_{uv}$ must be chosen proportional to e_{uv}: When Step 2C is applied to the successors v of the current node u, whose weights are $W[u]\, e_{uv}$, it then picks v with probability $\vec{p}_{uv} = e_{uv} / \sum_w e_{uw}$, and the new weight is $W[u] \sum_w e_{uw} = W[u]\, e_{uv}/\vec{p}_{uv}$, in accordance with the update formula (U_e).)

In contrast to Path Sampling, Algorithm TS as formulated does not allow an arbitrary probability distribution for selecting among the successors. Section 5.5 will show how an arbitrary nonuniform selection can be accommodated by generalizing the condensation step 2C.

It can happen that a node v is removed from A and later reinserted. For example, in the graph of Figure 1, after initially expanding v_1 into the set $A = \{v_2, v_3\}$, the algorithm could first expand v_3, and later v_2, reinserting v_3 into A. This is obviously a waste of effort, and it can be avoided by choosing the nodes in Step 2E in some topological order of the DAG. In many

applications, it is easy to obey this rule. For example, the permutation DAG is graded: All paths from the source to a node v have the same length. More generally, we may have some rank function $r\colon V \to \mathbb{Z}$ or $r\colon V \to \mathbb{R}$ such that $r(v) > r(u)$ for all arcs uv. When we have the graph of a polytope, oriented by some generic linear objective function f, then f can serve as a rank function for the nodes. In other instances, it may not be straightforward to find out if a node v is reachable from a node u. In any case, the correctness of the algorithm does not depend on any proper order in which the nodes are chosen.

Keeping multiple copies of the same node. In the above formulation of Step 2E, we check the presence of a node v in A before trying to insert it, but this is actually not necessary. We can also insert a fresh copy of v into A with a separate weight $W[v]$, (which is initialized to 0 as usual). Accordingly, we have to treat A as a *multiset*. When such a duplication is detected, it is of course expedient to condense multiple copies of the same node v, as a special case of Step 2C; but again, the correctness of the algorithm does not depend on this.

The general scheme of Team Sampling offers a lot of freedom. We describe and test two strategies that specialize this scheme, *Bucket Sampling* and *Crowd Sampling*, both inspired by methods that have been proposed for trees in the literature.

5.2 Bucket Sampling (Algorithm BS)

This strategy is modeled after the algorithm of Pang C. Chen [10] from 1992, which was proposed for trees under the name *heuristic sampling*; Knuth called it *Chen subset sampling* [24, Section 7.2.2.9, Algorithm C]. Nodes with similar characteristics are lumped together in a group S that is to be condensed. The idea is that, if the groups are formed in a sensible way, any element $u \in S$ is as good a representative of S as any other element.

Formally, the algorithm uses a *heuristic function* $h\colon V \to U$ from the nodes V to some universe U, which is used to identify the *buckets* to which the nodes are assigned. Ideally, the criterion by which nodes are grouped would be that the nodes have similar subtrees (when G is a tree) or similar sets of successors (when G is a DAG). However, these characteristics are usually not available or expensive to compute, so one has to settle for some easier "heuristic" characteristics. In our experiments, we have chosen the outdegree as the heuristic function.

In the original algorithm of Chen [10], a node and its ancestor node cannot be in one bucket, and what we call a bucket is called a *stratum*. This requirement, together with a proper order of processing the strata, ensures that the algorithm visits at most one node per stratum. With our relaxed notion of heuristic function, the stratum terminology is less appropriate, and we have chosen the term bucket instead.

Depending on the character of the universe U, the list of buckets can be organized in an array, a hash table, or a priority queue.

Conceptually, each bucket could store a list containing the (multi-)set of nodes v with the same value $h(v)$. *The algorithm repeatedly chooses some nonempty bucket, condenses it, and expands the chosen representative.*

However, the following considerations, which were already made for the original heuristic sampling method of [10], show that one only needs to store at most one node per bucket. Suppose we condense a set $S = \{u_1, u_2, u_3\}$, resulting in a representative $u' \in S$, and later condense the set $\{u', u_4, u_5, u_6\}$. It can be easily checked that the total effect is the same as condensing the set $\{u_1, u_2, u_3, u_4, u_5, u_6\}$ in one go. Thus, if we know that u_1, u_2, u_3 will eventually be condensed (possibly together with more elements that are not yet known), it makes sense to condense them right away, in order to save space.

Therefore, we store at most one node per bucket. In the expansion step, we generate successor nodes and insert them into their buckets. Whenever a node is inserted into a bucket that is already

occupied, we immediately condense the set of these two nodes. (This procedure corresponds to the *reservoir sampling* method of selecting a sample from a data stream.)

The resulting algorithm with the modified main loop is shown here:

Algorithm BS: Bucket Sampling

1. [Initialize] $A := \{s_0\}$; $W[s_0] := 1$; $X := 0$.

2. While A is nonempty, perform the following step:

BS-2E. [Expand successors] Choose an arbitrary node u of A, and remove it from A.
Set $X := X + W[u] \cdot c(u)$. // Augment X
For each successor v of u:
If A contains no node v' with $h(v) = h(v')$,
Insert v into A and set $W[v] := W[u]e_{uv}$.
Otherwise, i.e., if A contains a node v' with $h(v) = h(v')$,
[Condense v and v'] Compute $\bar{W} := W[v'] + W[u]e_{uv}$.
With probability $W[v']/\bar{W}$, set $W[v'] := \bar{W}$ (and ignore v).
With the complementary probability,
replace v' by v in A, and set $W[v] := \bar{W}$.

3. [Terminate] In the end, return X.

Experiments. We have tried this approach for the permutation DAG and the Klee–Minty cubes. As a heuristic function, we have taken the outdegree. In each step, we have selected the nonempty bucket with the largest outdegree and expanded the node stored in this bucket. The rationale behind this choice is that nodes closer to the root tend to have larger outdegree. For the permutation DAG, it would have been natural to use the level (rank) as additional information, but we have ignored this for simplicity.[3]

The results are shown in Tables 4 and 5. The number of iterations (reported as "#visited") is larger than for Path Sampling, and in addition, each visit incurs more work than for Path Sampling; therefore we went only up to $m = 10^6$ repetitions. One can see that Bucket Sampling with our very simple bucketing strategy improves the estimate. Bucket Sampling reduces the empirical standard deviation by a factor of about 2 to 12 in comparison with Algorithm P, but it remains larger than the mean.

We also report as "#generated" the total number of successor nodes that are generated and inserted into buckets; this quantity is representative of the total work of the algorithm. Our digraphs are regular graphs, of degree $n - 1$ for the permutation DAGs, and of degree n for the Klee–Minty cubes. On average, half of the incident arcs are outgoing arcs. Thus the number of generated nodes should be roughly the number of visited nodes times $(n-1)/2$ or $n/2$, respectively. The actual number of generated nodes ("#generated") is just slightly smaller than what one would expect from this calculation.

5.3 Crowd Sampling (Algorithm CS)

This strategy is inspired by the *stochastic enumeration* (SE) algorithm, which was proposed in 2013 by Reuven Y. Rubinstein [33] to count the number of nodes of a tree. The algorithm has a

[3]Some preliminary experiments (not reported here) suggest that bucketing by level *and* outdegree gives a substantial improvement: It reduces the standard deviation by a factor of 4 to 6, and it markedly reduces the number of visited nodes.

Permutations						
	$n = 8$			$n = 9$		
m	mean ± std. dev.	max.	$\hat{\chi}$	mean ± std. dev.	max.	$\hat{\chi}$
10^1	41,711.3 ± 68,536.3	2.12×10^5	0.50920	120,931.2 ± 1.24×10^5	4.52×10^5	0.37377
10^2	39,047.5 ± 1.40×10^5	1.36×10^6	0.34866	258,487.9 ± 9.36×10^5	8.60×10^6	0.33284
10^3	34,062.1 ± 89,284.0	1.73×10^6	0.05073	398,791.7 ± 3.63×10^6	1.08×10^8	0.27097
10^4	38,986.7 ± 1.34×10^5	5.97×10^6	0.01532	437,469.8 ± 1.12×10^7	1.09×10^9	0.25028
10^5	39,853.5 ± 2.02×10^5	4.61×10^7	0.01157	357,426.5 ± 4.90×10^6	1.09×10^9	0.03063
10^6	40,025.5 ± 2.55×10^5	1.28×10^8	0.00321	360,984.5 ± 5.31×10^6	2.67×10^9	0.00740
	40,320 #visited: avg = 84.4, max = 277 #generated: avg = 290.6			362,880 #visited: avg = 180.7, max = 763 #generated: avg = 690.5		

Table 4: Bucket Sampling for the permutation DAG.

Klee–Minty cubes						
	$n = 10$			$n = 12$		
m	mean ± std. dev.	max.	$\hat{\chi}$	mean ± std. dev.	max.	$\hat{\chi}$
10^1	426.2 ± 297.3	1.01×10^3	0.23671	2,912.5 ± 3,257.5	9.80×10^3	0.33636
10^2	1,002.6 ± 2,417.0	2.08×10^4	0.20746	2,805.3 ± 3,774.8	2.08×10^4	0.07424
10^3	975.0 ± 1,852.5	2.49×10^4	0.02554	4,418.8 ± 20,658.1	4.78×10^5	0.10808
10^4	970.3 ± 2,375.3	8.89×10^4	0.00916	3,838.8 ± 17,371.0	1.21×10^6	0.03150
10^5	1,035.6 ± 8,173.8	2.13×10^6	0.02057	3,982.0 ± 27,084.6	4.81×10^6	0.01209
10^6	1,024.6 ± 4,759.7	2.13×10^6	0.00208	4,106.5 ± 50,004.1	3.63×10^7	0.00884
	1,024 #visited: avg = 193.2, max = 1105 #generated: avg = 937.0			4,096 #visited: avg = 542.4, max = 4291 #generated: avg = 3092.3		

Table 5: Bucket Sampling for Klee–Minty cubes.

budget parameter B, imposing a limit on the number of nodes that we want to keep for further expansion. The algorithm consists of an alternating sequence of expansion and condensation phases. The main loop is as follows.

CS-2E [Expansion]
Expand every node of the current set A.
Let A' denote the resulting multi-set (or set, after identifying duplicates).

CS-2C [Condensation]
If $|A'| \leq B$, let $A := A'$.
Otherwise, arbitrarily partition A' into B sets $S_1 \cup \cdots \cup S_B$ of size $\lfloor \frac{|A'|}{B} \rfloor$ or $\lceil \frac{|A'|}{B} \rceil$. Condense each set S_i to a single element, and let A be the set of these B elements.

We call this method *Crowd Sampling*, because the samples are treated more like an unorganized crowd than a cooperating team.

The purpose of Step CS-2C is to reduce the size of the active set A' to B. The method we have proposed is a straightforward way to achieve this. In our implementation, we permute the set A' randomly and then partition it into sets of the required size. One can think of simpler methods, like a round-robin distribution in the order in which the elements happen to be stored in A', or more sophisticated methods, taking into account the weights, for example. Unbiasedness does not depend on the way how the partition is carried out.

Tables 6 and 7 show the results of this method with a budget of $B = 10$ elements. We also report the number of nodes that were saved by condensing multiple copies of the same node after

the expansion step ("#saved"). Compared to our Bucket Sampling experiments (Tables 4–5), we see that the standard deviation is further reduced, at the expense of a larger number of visited nodes. The number of visited nodes can be controlled by adjusting the parameter B.

There is another partial backtracking method, proposed by Purdom [31], which pursues a similar goal of examining every level of a tree at about the same frequency. It is discussed in Appendix C.

Permutations						
	$n = 8$			$n = 9$		
m	mean ± std. dev.	max.	$\hat{\chi}$	mean ± std. dev.	max.	$\hat{\chi}$
10^1	51,102.7 ± 43,393.4	1.56×10^5	0.30491	199,951.9 ± 1.45×10^5	4.06×10^5	0.20327
10^2	43,250.2 ± 33,729.6	1.56×10^5	0.03603	370,850.2 ± 3.97×10^5	2.05×10^6	0.05538
10^3	43,252.0 ± 39,430.2	3.70×10^5	0.00855	317,016.9 ± 3.77×10^5	3.08×10^6	0.00971
10^4	40,068.4 ± 35,080.8	7.40×10^5	0.00185	359,232.3 ± 5.80×10^5	1.84×10^7	0.00513
10^5	40,222.5 ± 35,171.4	8.77×10^5	0.00022	362,323.6 ± 5.76×10^5	2.37×10^7	0.00066
10^6	40,259.1 ± 35,177.0	1.37×10^6	0.00003	361,797.9 ± 5.81×10^5	7.45×10^7	0.00021
	40,320 #visited: avg = 261.1, max = 266 #generated: avg = 893.4 #saved: avg = 178.7 = 20.0%			362,880 #visited: avg = 341.8, max = 348 #generated: avg = 1329.4 #saved: avg = 240.4 = 18.1%		

Table 6: Crowd Sampling for the permutation DAG with a budget $B = 10$.

Klee–Minty cubes						
	$n = 10$			$n = 12$		
m	mean ± std. dev.	max.	$\hat{\chi}$	mean ± std. dev.	max.	$\hat{\chi}$
10^1	698.9 ± 362.5	1.49×10^3	0.21311	1,747.4 ± 1,241.1	4.00×10^3	0.22862
10^2	1,010.6 ± 1,053.9	6.45×10^3	0.06381	3,533.0 ± 6,814.3	5.70×10^4	0.16136
10^3	1,006.2 ± 1,586.2	2.99×10^4	0.02973	3,861.8 ± 11,011.9	2.55×10^5	0.06613
10^4	1,023.7 ± 1,559.8	5.64×10^4	0.00551	4,172.8 ± 12,451.9	5.04×10^5	0.01208
10^5	1,019.5 ± 1,446.9	7.24×10^4	0.00071	4,107.1 ± 11,862.7	8.52×10^5	0.00207
10^6	1,022.6 ± 1,490.0	2.40×10^5	0.00023	4,104.1 ± 13,739.7	3.46×10^6	0.00084
	1,024 #visited: avg = 734.0, max = 1381 #generated: avg = 2559.3 #saved: avg = 408.3 = 16.0%			4,096 #visited: avg = 971.0, max = 1880 #generated: avg = 4012.0 #saved: avg = 433.3 = 10.8%		

Table 7: Crowd Sampling for Klee–Minty cubes with a budget $B = 10$.

5.4 Correctness of the Team Sampling scheme

We use the notation $\mathcal{P}(v \to w)$ for the sum of all path weights from v to w:

$$\mathcal{P}(v \to w) = \sum_{\text{paths } P \text{ from } v \text{ to } w} e(P)$$

Lemma 7. *Suppose that at some stage in Algorithm TS, we have a current estimate X^{current} and a current active set A with weights $W[v]$ for $v \in A$. If we perform only expansion steps (Step 2E) until the end of the algorithm, then the value of the estimate X at termination can be predicted deterministically, and it is*

$$X^{\text{current}} + \sum_{v \in A} W[v] f_v, \tag{7}$$

for some factors f_v that depend only on the graph G, the costs $c(v)$, and the arc weights e_{vw}:

$$f_v := \sum_{w \in V} \mathcal{P}(v \to w) c(w) \tag{8}$$

In particular, an expansion step does not change the value of (7).

Proof. It is clear that the last sentence implies the whole lemma, since, at termination, $A = \emptyset$, and X^{current} is the final value of X.

Consider the expansion of a node $u \in A$, and let $v_1, \ldots, v_d$ be the $d^+(u)$ successors of u. It follows from the definition that f_u satisfies the relation

$$f_u = c(u) + \sum_{i=1}^{d^+(u)} e_{uv_i} f_{v_i}. \tag{9}$$

Expanding node $u \in A$ adds $c(u)W[u]$ to X^{current}, and it adds $e_{uv_i}W[u]$ to each $W[v_i]$, so the expression (7) increases by $W[u](c(u) + \sum_{i=1}^{d^+(u)} e_{uv_i} f_{v_i})$. On the other hand, u is removed from A, causing a decrease by $W[u]f_u$. By (9) these two changes cancel out. □

We now show that the value of (7) is left invariant also by a condensation step. In contrast to the expansion step (Lemma 7), this holds only in expectation:

Lemma 8. *After a condensation step, the expected value of $\sum_{v \in A} W[v]f_v$ is the same as before.*

Proof. Condensing replaces a set S by one of its members $u \in S$, where u is chosen with probability $W[u]/\sum_{v \in S} W[v]$. Let $\hat{A} = (A \setminus S) \cup \{u\}$ be the new active set, and let W' denote the new weights. The new weight of u is $W'[u] = \sum_{v \in S} W[v]$. The weights in $A \setminus S$ are unchanged. Thus we get

$$E\left[\sum_{v \in \hat{A}} W'[v]f_v\right] = \sum_{v \in A \setminus S} W[v]f_v + \sum_{u \in S}\Big(\Pr[u \text{ is chosen}] \times W'[u]f_u\Big) \tag{10}$$

$$= \sum_{v \in A \setminus S} W[v]f_v + \sum_{u \in S}\left(\frac{W[u]}{\sum_{v \in S} W[v]} \times \sum_{v \in S} W[v]f_u\right) \tag{11}$$

$$= \sum_{v \in A \setminus S} W[v]f_v + \sum_{u \in S} W[u]f_u = \sum_{v \in A} W[v]f_v, \tag{12}$$

as claimed. □

Theorem 9. *Algorithm TS produces an estimate with the same expectation as Algorithm P:*

$$E[X] = c_e(G) = \sum_{v \in V} \mathcal{P}(s_0 \to v) c(v)$$

Proof. Lemmas 7 and 8 together imply that, at any time during the algorithm,

$$E[X^{\text{final}}] = X^{\text{current}} + \sum_{v \in A} W[v]f_v.$$

If we apply this to the starting situation of Algorithm TS, with $A = \{s_0\}$, $W[s_0] = 1$, and $X^{\text{current}} = 0$, we get $E[X^{\text{final}}] = f_{s_0}$, which equals the claimed quantity. □

As a side result of this analysis, we get a combinatorial interpretation of the expectation of the random variables F_v introduced in Section 3 to compute the variance of the estimate X. Comparing (9) with (6) and taking note of the fact that μ_j in (6) stands for $E[F_{v_j}]$, we see that f_v and $E[F_v]$ satisfy the same recurrence relation. If v is a sink, (9) and (6) specify the same values $f_v = E[F_v] = c(v)$, which can be used as initial conditions. Thus:

Proposition 10. *The expected value of F_v equals f_v.* □

5.5 Generalized Condensation

The Team Sampling algorithm shown in Section 5.1 proposes a specific method for the Condensation Step (choice of probabilities and weight update). We will see that this is just one of many possibilities.

As mentioned, Crowd Sampling was modeled after the Stochastic Enumeration algorithm of Rubinstein [33], which works for trees. For a tree, the algorithm becomes very simple: In the expansion step, all backward proportions $\overleftarrow{q}_{uv}$ are 1, and hence the children just get the same weights as their parents. If the resulting set A' of children is too big, it is simply replaced by a random sample of size B, and the weight is accordingly multiplied by $|A'|/B$. All nodes have the same weight.

Our Crowd Sampling method mimics the behavior of this algorithm only when $|A'|$ is a multiple of B and the partition of A' into equal-size subsets S_i is random. Otherwise, Crowd Sampling generates nodes of different weights, introducing irregularities into the process. In fact, for DAGs, nodes of different weights appear anyway, for another reason. For a set with weights, the meaning of "a random sample of size B" is not obvious; see the survey [14] for two different potential interpretations. Let us take an engineering approach and formulate what is required to ensure that the estimate remains unbiased. It will turn out that this actually leaves a lot of freedom in performing the condensation step.

Assume that the set S contains k nodes with weights $W_1, W_2, \ldots, W_k$. Suppose we decide that Step 2C should randomly select the i-th node with probability p_i, and assign weight W_i' to it in case it is selected. If we look at the derivation (10–12) in the proof of Lemma 8, we see that what we need for unbiasedness is the equation

$$p_1 W_1' f_1 + \cdots + p_k W_k' f_k = W_1 f_1 + \cdots + W_k f_k. \tag{13}$$

Since we don't know the values f_i, this equation should hold for all $f_1, \ldots, f_k$. Comparing coefficients leads to k equations

$$p_i W_i' = W_i, \tag{14}$$

for $i = 1, \ldots, k$. Together with the equation $\sum_{i=1}^{k} p_i = 1$, this gives $k+1$ equations for $2k$ unknowns p_i, W_i', which is far from enough to define a unique solution.[4]

The basic condensation step of Algorithm TS chooses $W_i' = \bar{W} := W_1 + \cdots + W_k$ and $p_i = W_i/\bar{W}$, but it is not clear why this choice should be preferable over other choices. In fact, we can select arbitrary probabilities p_i (for example, uniform probabilities) and set $W_i' := W_i/p_i$. In this way, we can reinstate the forward proportions of Algorithm P, which Algorithm TS in its original form could not handle.

The generalized condensation step for Algorithm TS is thus as follows:

2C′. [Condense] Choose an arbitrary subset $S = \{u_1, u_2, \ldots, u_k\}$ of A with $k \geq 2$ elements.
Choose positive probabilities p_i with $p_1 + p_2 + \cdots + p_k = 1$.
Pick a random node u_i from S according to this distribution.
Replace S in A by the single element u_i, and set $W[u_i] := W[u_i]/p_i$.

5.6 The variance for Team Sampling strategies

For particular strategies that specialize the Team Sampling procedure, it should in principle be possible to extend the results of Section 3 and compute the exact variance of the estimate X if the whole graph G with all forward and backward proportions is given. The general idea is that each condensation step contributes to the variance, because it replaces a set S by a single sample.

[4] We may even permit $\sum p_i < 1$, effectively throwing away the sample with probability $1 - \sum p_i$. In the single path variation (Algorithm P), the algorithm gets a chance to finish early and take a rest. This comes, of course, at the expense of a larger fluctuation of the estimate.

Knuth [24, Section 7.2.2.9, Theorem V] has given a method to compute the variance for Chen's version [10] of Bucket Sampling, where the heuristic function partitions the nodes into strata in an acyclic way. It would be interesting to get a similar analysis for Crowd Sampling.

5.7 Relation to dynamic programming

Many counting algorithms are based on *dynamic programming*, or the *transfer-matrix method* as it is sometimes called depending on the implementation. We show that these algorithms can be seen as a special case of Bucket Sampling.

We illustrate this with the problem of counting directed lattice animals, following Conway [11]. A *directed animal* (see Figure 4) is a polyomino in the first quadrant such that every occupied cell, except the cell at the bottom-left corner, has an occupied cell to its left or at its bottom (or both). Directed animals are theoretically well-understood [15], and they are chosen here for illustration only.

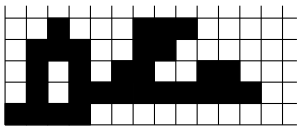


Figure 4: A directed animal with 29 cells

Figure 5a shows an *enumeration tree* T for the directed animals. The animals have been turned by 135° and grow from top to bottom. The black cells are the cells that are already part of the animal, and the shaded cells are the adjacent cells below the chosen cells whose inclusion has not been decided yet. At each tree node, we take the leftmost undecided cell in the highest row (shaded in blue) and decide whether it is excluded (marked by a cross) or included (turned black). If there is only a single shaded cell, it has to be included. In this way, we try out all possibilities of generating a directed animal. We stop when we have 4 black cells, and hence the 13 leaves of this tree represent the directed animals of size 4.

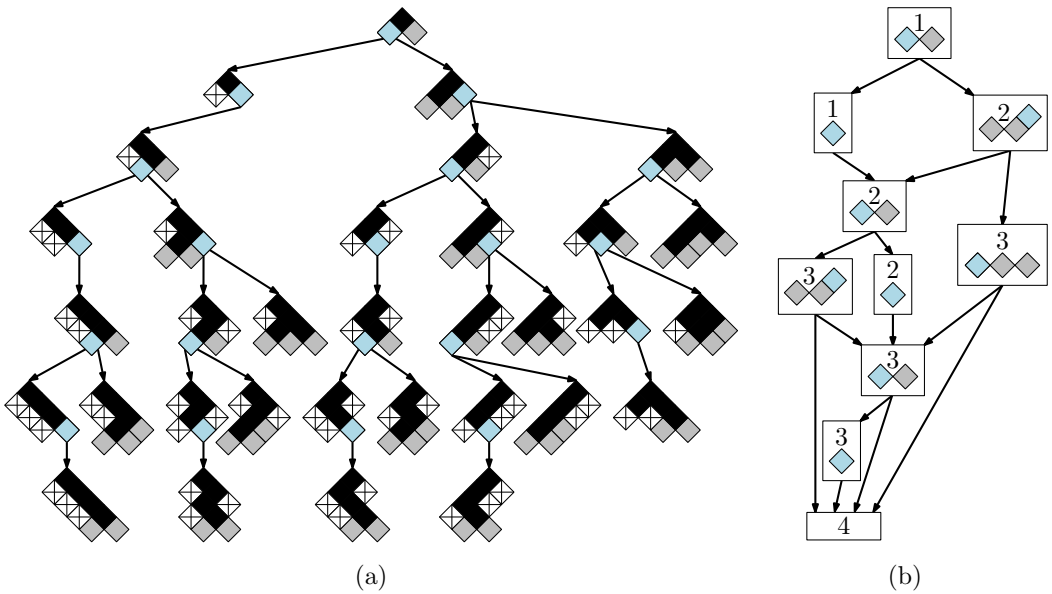


Figure 5: (a) an enumeration tree T for directed animals and (b) the associated dynamic-programming DAG G, adapted from Conway [11, Fig. 1 and 2]

The main idea of dynamic programming is that we don't need to remember the whole animal as it is built: For determining how to continue, it suffices to know the *pattern* of the shaded (grey and blue) cells and the *number* of black cells. Nodes for which this information agrees have

equivalent subtrees. By merging such nodes, we arrive at the DAG G shown in Figure 5b. The directed animals correspond now to the source-sink paths in G.

The DAG G is smaller than the tree T from which it originated. (For larger examples than the one shown in the picture, the reduction in size is more impressive.) However, for the single-path algorithm (Algorithm P), it makes *absolutely no difference* whether it moves in the tree T to count leaves according to Theorem 1 or in the DAG G with update rule (U_1) to count source-sink paths according to Theorem 2. (In this spirit, Conway's reference to the DAG in [11, Fig. 2] as a "tree" is perhaps not completely off-track.)

We can use Bucket Sampling with a heuristic function h that maps the nodes of the tree T to the corresponding nodes in the DAG G. It turns out that, in this way, we arrive essentially at the classic dynamic-programming algorithm for counting objects by the DAG: We scan the DAG top-down, adding the weight of each node to all its children. Since only nodes with equal subtrees are thrown together in one bucket, the algorithm gives the exact result.

But the possibilities of Bucket Sampling go beyond this: We are allowed to merge nodes whose subtrees are different (but hopefully at least similar). The transition from the tree to the DAG, which is crucial for the speed-up in dynamic programming, is, however, not essential for Bucket Sampling, because it is always implicitly included. Perhaps, advantages of working with a DAG instead of an enumeration tree will show up in connection with other Team Sampling strategies.

6 Extensions and potential applications

6.1 Extension to arc costs

For a graph where a cost $c(uv)$ is associated to each arc uv, the extension to estimating the total sum of all arc costs is straightforward: When proceeding from u to v, the algorithm adds $W \cdot c(uv)/\vec{p}_{uv}$ to X. Conceptually, we could simply subdivide every arc and let the newly inserted nodes take over the role of the arcs.

6.2 Some applications

We list a few applications where it might be promising to try some of our estimation algorithms. The number of vertices of a polytope, to be discussed in the first subsection, appears very natural; most of the remaining applications are more speculative.

6.2.1 Vertices of a polytope

For a polytope, a linear objective function induces an orientation of the graph of the polytope: It directs each edge in the direction of improvement of the objective function, provided that the objective function is sufficiently generic and does not assign the same value to adjacent vertices.

As mentioned in Section 1, Avis and Devroye have used the tree version of Algorithm P to estimate the number of vertices of a polytope [3]. The simplex algorithm generates a path from each vertex of a polytope to the optimum vertex, and thereby defines a spanning tree of the vertices. This spanning tree is at the core of the *reverse search* method for enumerating all vertices, and it is the tree to which Avis and Devroye apply Algorithm P. Salomone, Vaisman, and Kroese [34] have applied the Stochastic Enumeration method (which we have called Crowd Sampling) to this tree.

However, the graph of a polytope whose edges are oriented by an objective function is naturally a DAG.[5] We intend to compare our approach against the tree-based estimation methods of [3] and [34].

[5] Every Klee–Minty cube is of course a polytope, but also the permutation DAG is obtainable as the graph of a polytope, namely the *permutahedron*: The vertex coordinates of the permutahedron are given by the inverse permutations of the permutations that form the nodes of the permutation DAG as described here. For any generic objective function, the directed graph of the permutahedron edges is isomorphic to the permutation DAG.

When pivoting from vertex to vertex by the simplex algorithm, one has to take into account an unpleasant technicality: The objects that the simplex algorithm visits are the *feasible bases*, and for polytopes that are not simple, a vertex can correspond to several bases. The standard way to treat such *degenerate* or nonsimple polytopes, where more than n facets meet in a vertex, is to perturb the constraints, resulting in a simple polytope, and count only one of the many vertices that correspond to a vertex of the original polytope. See [3] for details.

6.2.2 Triangulations of a planar point set

Alvarez and Seidel [2] have given an algorithm with runtime $O(2^n n^2)$ for counting the triangulations of an n-point set in the plane. They model triangulations as source-sink paths in a DAG with $n2^{n-3}$ nodes. Our algorithms can therefore be readily applied. The quality of the resulting approximation remains to be explored. The state of the art about approximate counting of triangulations is rather weak: There is only a subexponential-time approximate counting method with a subexponential multiplicative error [1]. The current fastest algorithm for exact counting takes time $n^{O(\sqrt{n})}$ [29].

6.2.3 Linear extensions of a two-dimensional partial order

If we start from an arbitrary permutation $\pi = (\pi_1, \ldots, \pi_n)$ in the permutation DAG, the reachable nodes are the linear extensions of a two-dimensional partial order $\prec$, which is defined by the rule $i \prec j :\Leftrightarrow i < j \wedge \pi_i < \pi_j$. Thus our path sampling methods can be used to estimate the number of linear extensions of such a partial order. The computation of indegrees must be adjusted to count only those predecessor permutations that can be reached from π.

Counting the linear extensions is #P-complete, even for a two-dimensional partial order, as proved by Dittmer and Pak [13, Theorem 1.3].[6] In any case, there already exists a fully polynomial randomized approximation scheme for the number of linear extensions of an arbitrary poset, because the number of linear extensions is directly related to the volume of the order polytope, and the polytope volume can be approximated by Markov chain Monte Carlo methods, see for example [8, Section 3] for details and references. Our path sampling methods might lead to alternative approximation schemes that are simpler.

6.2.4 Pattern-avoiding permutations

A permutation τ *contains* a permutation $\sigma = (\sigma_1, \ldots, \sigma_k)$, the *pattern*, if τ has a subsequence of length k whose elements are in the same relative order as the elements of σ; otherwise, τ *avoids* σ. For example, the permutation 14253 from Figure 2 contains the pattern 1324, in the form of the subsequence 1425, whereas its predecessor 14523 on the path avoids 1324. By the Marcus–Tardos theorem, the number of permutations of length n avoiding a fixed pattern grows only exponentially in n [27]. For the patterns of length 3, these numbers are the Catalan numbers, and for length 4, there are three essentially different possibilities, so-called Wilf-equivalence classes, represented by the patterns 1234, 1342, and 1324 [37]. The 1324-avoiding permutations are the only ones among them for which no formula is known, and they have become a notorious benchmark problem for counting algorithms. Their numbers are known exactly up to $n = 50$ [12], and the exponential growth rate is estimated as 11.600 ± 0.003 by series analysis of these numbers, while the best proven bounds are 10.27 and 13.5 [7]. Markov chain Monte Carlo methods have been used to sample random 1324-avoiding permutations [25].

The class of σ-avoiding permutations is closed under taking subsequences in the following sense: If we delete an element from a σ-avoiding permutation and renumber the remaining elements consecutively, the result still avoids σ. Hence, the σ-avoiding permutations of length at most n form a DAG with the singleton permutation 1 as the source, in which the successors of

[6] There is a subsequent journal article by the same authors with the same title, which, however, omits this result and does not even mention it.

a permutation π of length $m < n$ are the σ-avoiding permutations of length $m+1$ obtained by inserting a half-integer element at an arbitrary position and renumbering. Every node can be reached from the source. Moreover, assuming $k \geq 2$, one can always append either a new largest or smallest element at the end without creating an occurrence of σ. Therefore, every source-sink path ends at length n. With costs $c(v) = 1$ for the permutations of length n and $c(v) = 0$ otherwise, Algorithm P estimates their number. The indegree is easy to compute and requires no test for pattern containment: Deleting either of two adjacent elements whose values differ by 1 gives the same result, and one can show that these are the only coincidences. Thus, for a permutation τ of length m,

$$d^-(\tau) = m - \big|\{\, i : |\tau_i - \tau_{i+1}| = 1 \,\}\big|.$$

For the outdegree, one must try the $(m+1)^2$ ways of inserting a new element into τ, discard duplicates (by the same rule), and check whether the new element creates an occurrence of σ.

If one only allows the insertion of a new *largest* element, one gets instead a tree, the so-called *generating tree* [38], to which the original tree version of Algorithm P is applicable. It would be interesting to see which of the two settings (tree or DAG) leads to better estimates. The exact counting algorithms for 1324-avoiding permutations are, in essence, dynamic-programming algorithms that merge nodes of the generating tree whose subtrees are equivalent [28, 11]. As discussed in Section 5.7 for directed animals, Bucket Sampling offers a way to relax this: nodes whose subtrees are only "similar" can be put into the same bucket. Since the exact counts up to $n = 50$ are available for comparison, the 1324-avoiding permutations are an ideal testing ground for our methods, which could then be applied to get estimates for larger n, or to estimate the counts for other patterns or for sets of several forbidden patterns, for which no specialized algorithms have been designed.

6.2.5 Polyominoes

Polyominoes are another classic counting problem of this kind; the directed animals of Section 5.7 are a special type of polyominoes. As for the 1324-avoiding permutations, the record exact counts for various types of polyominoes have been obtained by dynamic programming, known in this area as the *transfer-matrix* method, which avoids enumerating the polyominoes individually, see [5, 6] for recent advances, and [11] for an overview that covers both problems. As discussed in Section 5.7, such algorithms can be interpreted as accumulating (possibly weighted) source-sink paths in a DAG. They are run on huge DAGs, exhausting the limits of computer time and memory. With our approach, the same ideas can be leveraged to get estimates far beyond the polyomino sizes for which exact computation is feasible. This might be an alternative to Markov-chain based sampling techniques [35, 19]. With appropriate weights $c(v)$, one can also gather statistics about various properties of polyominoes.

6.2.6 Maximal sets of an independence system

Honcharov, Sarıyüce, Laishram, and Soundarajan [18] have independently discovered the generalization of Algorithm P from trees to DAGs for a special graph problem: They used it to estimate the number of *skeletal cores* of a graph, which are minimal subgraphs that preserve certain properties. More precisely, they have the property that every vertex keeps its original *core number*, where the core number of a vertex v is the largest k such that v is contained in a subgraph of minimum degree k.

They apply Algorithm P to a DAG whose nodes are subgraphs and whose arcs correspond to edge deletions. The sets of edges that can be deleted form an *independence system*: a family of sets that is closed under taking subsets. A skeletal core corresponds to a *maximal* set in this family (a *basis* of the independence system).

In this setting, the DAG is layered, and a node at distance j from the source has indegree j. This regular structure makes the algorithm particularly easy. Instead of dividing by the indegree

at every step, Honcharov et al. divide the final weight by $j!$, where j is the number of deleted edges, to account for the $j!$ different paths that reach the same sink. This gives exactly the same result as Algorithm P with the standard proportions and $c(v) = 1$ for the sinks.

6.3 Graphs with cycles

Marchetti-Spaccamela [26] has proposed an interesting extension of Algorithm P to arbitrary digraphs. From each node that is visited, this method repeatedly starts a random path in the reverse direction of the arcs and follows it until it arrives at a node that already lies on that path or at the source. The number of trials necessary to reach the latter outcome is used to adjust the estimate. Since the probability of this event can be very low, the method requires many samples, and it is practical only in limited settings.

7 Conclusion

We have proposed Team Sampling, a general scheme for obtaining unbiased estimates of the size of a DAG with a team of cooperating agents, and we have tried two specializations, Bucket Sampling and Crowd Sampling, which carry over methods of Chen [10] and Rubinstein [33] from trees to DAGs. There is ample opportunity to come up with more ideas and try them out; we have barely scratched the surface with our experiments. For example, one can combine the ideas of Bucket Sampling and Crowd Sampling, allowing up to B nodes to occupy one bucket.

7.1 Choice of forward and backward proportions

For the choice of forward proportions $\vec{p}_{uv}$, Importance Sampling gives some guidelines about what one wants to achieve (see Section 4).

If the backward proportions $\overleftarrow{q}_{uv}$ are nonnegative, they could of course be interpreted as transition probabilities for a random walk in the reverse graph. Nevertheless, their role in the algorithm is not understood at all. One may even allow them to become negative or zero: $\overleftarrow{q}_{uv} = 0$ effectively means that the sample is ignored when the algorithm traverses the arc uv. Negative proportions $\overleftarrow{q}_{uv} < 0$ are mysterious.

7.2 Provable approximation quality?

Vaisman and Kroese showed that Stochastic Enumeration (i.e., Crowd Sampling) is a fully polynomial randomized approximation scheme (FPRAS) for counting the number of nodes of trees, for *most* trees in a certain (rather restricted) class of random trees [36, Theorem 4.3[7]]. It would be interesting to see whether Path Sampling or Team Sampling leads to an FPRAS for some combinatorial counting problems. Some candidate problems were listed in Section 6.2.

Acknowledgment. The ideas of this paper were inspired by discussions initiated at the workshop on *Computational Polyhedral Geometry*, which took place on June 26, 2025 as part of the Computational Geometry Week (CG Week) in Kanazawa, Japan.[8] I thank Laura Casabella and Dante Luber for organizing this workshop, and the other participants for the lively discussions.

I thank Bruno Scherrer for making me aware that the idea of Algorithm P, which I developed independently, had been anticipated a long time ago by Leonard Pitt [30].

[7] In the statement of this theorem, the budget parameter B for the algorithm is not specified. It must be set to kB', where B' is defined in [36, Theorem 4.1, p. 52], depending polynomially on the tree height and the maximum degree k, as well as the parameters of the random tree model (confirmed by Radislav Vaisman, personal communication, July 2025).

[8] `https://socg25.github.io/workshops.html`

AI usage statement. The AI assistant Claude from Anthropic has been used to check the manuscript for consistency and correctness, and to suggest improved formulations in many places. The AI assistant pointed the author to Kitahara and Mizuno [20], where an earlier version of a Klee–Minty cube with integral vertices was used. Section 6.2.4 has been generated by the AI assistant from a stub after the author had forgotten what he intended to write there. The programs were initially written and the test runs were carried out, alas, without any AI assistance. The AI assistant found a programming error, by which Table 9 in Appendix A reported results of the same experiment as Table 8, and two inconsistencies between the programs and the descriptions in the text; all these mistakes have been corrected. The author assumes the responsibility for the results.

References

[1] Victor Alvarez, Karl Bringmann, Saurabh Ray, and Raimund Seidel. Counting triangulations and other crossing-free structures approximately. *Computational Geometry*, 48(5):386–397, 2015. Special Issue on the 25th Canadian Conference on Computational Geometry (CCCG). doi:10.1016/j.comgeo.2014.12.006.

[2] Victor Alvarez and Raimund Seidel. A simple aggregative algorithm for counting triangulations of planar point sets and related problems. In *Proceedings of the 29th Annual Symposium on Computational Geometry (SoCG'13)*, pages 1–8, New York, NY, USA, 2013. Association for Computing Machinery. doi:10.1145/2462356.2462392.

[3] David Avis and Luc Devroye. Estimating the number of vertices of a polyhedron. *Information Processing Letters*, 73(3):137–143, 2000. doi:10.1016/S0020-0190(00)00011-9.

[4] József Balogh and Robin Pemantle. The Klee–Minty random edge chain moves with linear speed. *Random Structures & Algorithms*, 30(4):464–483, 2007. doi:10.1002/rsa.20127.

[5] Gill Barequet and Gil Ben-Shachar. Counting polyominoes, revisited. In *Proceedings of the 2024 Symposium on Algorithm Engineering and Experiments (ALENEX)*, pages 133–143. SIAM, 2024. doi:10.1137/1.9781611977929.10.

[6] Gill Barequet and Gil Ben-Shachar. Counting polyominoes, revisited. *Algorithmica*, 88:51 (20 pp.), 2026. doi:10.1007/s00453-026-01392-6.

[7] David Bevan, Robert Brignall, Andrew Elvey Price, and Jay Pantone. A structural characterisation of Av(1324) and new bounds on its growth rate. *European Journal of Combinatorics*, 88:103115 (29 pp.), 2020. arXiv:1711.10325, doi:10.1016/j.ejc.2020.103115.

[8] Graham Brightwell and Peter Winkler. Counting linear extensions. *Order*, 8:225–242, 1991. doi:10.1007/BF00383444.

[9] Sourav Chatterjee and Persi Diaconis. The sample size required in importance sampling. *Ann. Appl. Probab.*, 28(2):1099–1135, 2018. doi:10.1214/17-AAP1326.

[10] Pang C. Chen. Heuristic sampling: A method for predicting the performance of tree searching programs. *SIAM Journal on Computing*, 21(2):295–315, 1992. doi:10.1137/0221022.

[11] Andrew R. Conway. The design of efficient dynamic programming and transfer matrix enumeration algorithms. *Journal of Physics A: Mathematical and Theoretical*, 50(35):353001 (28 pp.), August 2017. arXiv:1610.09806, doi:10.1088/1751-8121/aa8120.

[12] Andrew R. Conway, Anthony J. Guttmann, and Paul Zinn-Justin. 1324-avoiding permutations revisited. *Advances in Applied Mathematics*, 96:312–333, 2018. arXiv:1709.01248, doi:10.1016/j.aam.2018.01.002.

[13] Samuel Dittmer and Igor Pak. Counting linear extensions of restricted posets, 2018. arXiv:1802.06312.

[14] Pavlos S. Efraimidis. Weighted random sampling over data streams. In Christos Zaroliagis, Grammati Pantziou, and Spyros Kontogiannis, editors, *Algorithms, Probability, Networks, and Games: Scientific Papers and Essays Dedicated to Paul G. Spirakis on the Occasion of His 60th Birthday*, volume 9295 of *Lecture Notes in Computer Science*, pages 183–195. Springer International Publishing, 2015. arXiv:1012.0256, doi:10.1007/978-3-319-24024-4_12.

[15] D. Gouyou-Beauchamps and G. Viennot. Equivalence of the two-dimensional directed animal problem to a one-dimensional path problem. *Advances in Applied Mathematics*, 9(3):334–357, 1988. doi:10.1016/0196-8858(88)90017-6.

[16] Bernd Gärtner, Martin Henk, and Günter M. Ziegler. Randomized simplex algorithms on Klee-Minty cubes. *Combinatorica*, 18(3):349–372, 1998. doi:10.1007/PL00009827.

[17] Marshall Hall and D. E. Knuth. Combinatorial analysis and computers. *The American Mathematical Monthly*, 72(2):21–28, 1965. doi:10.2307/2313307.

[18] Danylo Honcharov, Ahmet Erdem Sarıyüce, Ricky Laishram, and Sucheta Soundarajan. Skeletal cores and graph resilience. In Danai Koutra, Claudia Plant, Manuel Gomez Rodriguez, Elena Baralis, and Francesco Bonchi, editors, *Machine Learning and Knowledge Discovery in Databases: Research Track (ECML PKDD 2023), Turin, Italy, September 2023*, volume 14171 of *Lecture Notes in Computer Science*, pages 293–308. Springer, 2023. doi:10.1007/978-3-031-43418-1_18.

[19] Hsiao-Ping Hsu, Walter Nadler, and Peter Grassberger. Simulations of lattice animals and trees. *Journal of Physics A: Mathematical and General*, 38(4):775–806, January 2005. arXiv:cond-mat/0408061, doi:10.1088/0305-4470/38/4/001.

[20] Tomonari Kitahara and Shinji Mizuno. Klee–Minty's LP and upper bounds for Dantzig's simplex method. *Operations Research Letters*, 39(2):88–91, 2011. doi:10.1016/j.orl.2011.01.003.

[21] Victor Klee and G. J. Minty. How good is the simplex algorithm? In O. Shisha, editor, *Inequalities III*, pages 159–175. Academic Press, New York, 1972.

[22] Donald E. Knuth. Estimating the efficiency of backtrack programs. *Mathematics of Computation*, 29:122–136, 1975. URL: https://www.ams.org/journals/mcom/1975-29-129/S0025-5718-1975-0373371-6/.

[23] Donald E. Knuth. *Combinatorial Algorithms, Part 2*, volume 4B of *The Art of Computer Programming*. Addison-Wesley, 2023.

[24] Donald E. Knuth. *Combinatorial Algorithms, Part 3*, volume 4C of *The Art of Computer Programming*. Addison-Wesley, 2027+. In preparation. Draft of Section 7.2.2.9, "Estimating Backtrack Costs" at https://cs.stanford.edu/~knuth/fasc9c.ps.gz, version March 27, 2022.

[25] Neal Madras and Hailong Liu. Random pattern-avoiding permutations. In Manuel E. Lladser et al., editors, *Algorithmic Probability and Combinatorics*, volume 520 of *Contemporary Mathematics*, pages 173–194. American Mathematical Society, Providence, RI, 2010.

[26] A. Marchetti-Spaccamela. On the estimate of the size of a directed graph. In J. van Leeuwen, editor, *Graph-Theoretic Concepts in Computer Science*, volume 344 of *Lecture Notes in Computer Science*, pages 317–326, Berlin, Heidelberg, 1989. Springer Berlin Heidelberg. doi:10.1007/3-540-50728-0_54.

[27] Adam Marcus and Gábor Tardos. Excluded permutation matrices and the Stanley–Wilf conjecture. *Journal of Combinatorial Theory, Series A*, 107(1):153–160, 2004. doi:10.1016/j.jcta.2004.04.002.

[28] Darko Marinov and Radoš Radoičić. Counting 1324-avoiding permutations. *Electronic Journal of Combinatorics*, 9(2):#R13, 2003. doi:10.37236/1685.

[29] Dániel Marx and Tillmann Miltzow. Peeling and nibbling the cactus: Subexponential-time algorithms for counting triangulations and related problems. In Sándor P. Fekete and Anna Lubiw, editors, *32nd International Symposium on Computational Geometry, SoCG 2016, Boston, MA, USA, June 14-18, 2016*, LIPIcs, pages 52:1–52:16. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2016. doi:10.4230/LIPICS.SOCG.2016.52.

[30] Leonard Pitt. A note on extending Knuth's tree estimator to directed acyclic graphs. *Information Processing Letters*, 24(3):203–206, 1987. doi:10.1016/0020-0190(87)90187-6.

[31] Paul W. Purdom. Tree size by partial backtracking. *SIAM Journal on Computing*, 7(4):481–491, 1978. doi:10.1137/0207038.

[32] Günter Rote. NumPSLA – an experimental research tool for pseudoline arrangements and order types. In Jan Kratochvíl and Giuseppe Liotta, editors, *41st European Workshop on Computational Geometry (EuroCG 2025)*, pages 18:1–18:8, April 2025. arXiv:2503.02336.

[33] Reuven Rubinstein. Stochastic enumeration method for counting NP-hard problems. *Methodology and Computing in Applied Probability*, 15:249–291, 2013. doi:10.1007/s11009-011-9242-y.

[34] Robert Salomone, Radislav Vaisman, and Dirk Kroese. Estimating the number of vertices in convex polytopes. In *Proceedings of the 4th Annual International Conference on Operations Research and Statistics (ORS 2016), Singapore, January 2016*, pages 96–105. Global Science and Technology Forum, Singapore, 2016. doi:10.5176/2251-1938_ORS16.25.

[35] D. Stauffer. Monte Carlo study of density profile, radius, and perimeter for percolation clusters and lattice animals. *Phys. Rev. Lett.*, 41:1333–1336, November 1978. doi:10.1103/PhysRevLett.41.1333.

[36] Radislav Vaisman and Dirk P. Kroese. Stochastic enumeration method for counting trees. *Methodology and Computing in Applied Probability*, 19:31–73, 2017. doi:10.1007/s11009-015-9457-4.

[37] Vincent Vatter. Permutation classes. In Miklós Bóna, editor, *Handbook of Enumerative Combinatorics*, chapter 12, pages 753–833. CRC Press, Boca Raton, FL, 2015. arXiv:1409.5159, doi:10.1201/b18255-18.

[38] Julian West. Generating trees and the Catalan and Schröder numbers. *Discrete Mathematics*, 146(1–3):247–262, 1995. doi:10.1016/0012-365X(94)00067-1.

A Importance Sampling for Klee–Minty cubes

As mentioned in Section 4, we have tried Importance Sampling by outdegree also for Klee–Minty cubes. In comparison to Table 2, the results of Table 8 show a substantial improvement in the theoretical standard deviation, by several orders of magnitude. Like for permutations, the comparison in terms of empirical behavior is not at all clear. Even with 10 million samples, the fluctuations are too high to make a reasonable assessment.

Klee–Minty cubes						
	$n = 10$			$n = 12$		
m	mean ± std. dev.	max.	$\hat{\chi}$	mean ± std. dev.	max.	$\hat{\chi}$
10^1	65.0 ± 46.7	1.45×10^2	0.22366	503.5 ± 668.7	1.99×10^3	0.39488
10^2	$500.7 \pm 1{,}279.9$	7.67×10^3	0.15319	$685.5 \pm 1{,}676.9$	1.05×10^4	0.15256
10^3	$649.7 \pm 3{,}306.4$	5.54×10^4	0.08534	$1{,}854.5 \pm 15{,}942.5$	3.76×10^5	0.20282
10^4	$880.2 \pm 11{,}958.6$	8.46×10^5	0.09610	$2{,}418.5 \pm 33{,}458.9$	2.61×10^6	0.10777
10^5	$910.0 \pm 37{,}608.7$	1.16×10^7	0.12699	$6{,}892.9 \pm 1.23\times10^6$	3.86×10^8	0.55991
10^6	$983.9 \pm 87{,}220.8$	8.33×10^7	0.08466	$3{,}969.2 \pm 5.15\times10^5$	3.86×10^8	0.09723
10^7	$1{,}006.4 \pm 1.45\times10^5$	4.38×10^8	0.04347	$3{,}737.7 \pm 4.70\times10^5$	7.03×10^8	0.01881
	$1{,}024 \quad \pm 2.141\times10^6$			$4{,}096 \quad \pm 7.762\times10^{10}$		
	#visited: avg = 22.4, max = 86			#visited: avg = 29.2, max = 104		

Table 8: Algorithm P for Klee–Minty cubes with Importance Sampling by outdegree

A.1 Importance Sampling by the size of the reachable set

As mentioned, when Path Sampling is applied to a rooted tree, the ideal forward proportions are known: The probabilities $\vec{p}_{uv}$ should be proportional to the cost of the subtrees rooted at the children v of node u. For DAGs, it is not clear whether such a perfect choice of forward proportions that completely eliminates variations of X exists. As a substitute for the size of the subtree rooted at v, we may take the size R_v of the node set reachable from v.

For Klee–Minty cubes, we are in the lucky situation that the size R_v of the reachable set is readily available: The Klee–Minty cube of dimension n has a directed Hamilton path, in which the last coordinate x_n increases in an arithmetic progression $0, 1, 2, \ldots, 2^n - 1$, see Figure 3. Thus, $R_v = 2^n - x_n(v)$.

We made two experiments where R_v is taken into account for computing the forward proportions. We emphasize that this is not suitable in practical applications: If the size of the reachable set were known, there would be no need to compute an estimate by running a sampling algorithm. However, the experiment teaches us what we might be looking for in the choice of forward proportions.

In the first experiment (Table 9), we simply chose the probabilities $\vec{p}_{uv}$ for the successors v of a node u proportional to R_v. We see no improvement over simply using the outdegree. For $n = 12$, both the observed and the theoretical standard deviation are even about one order of magnitude larger.

In the second experiment (Table 10), we took into account the fact that a node v with large indegree is visited from several predecessors. We therefore chose the probabilities proportional to $R_v/d^-(v)$. Here we get much better estimates, and we see a substantial reduction of the standard deviation. While these weights are not perfect, they indicate the direction in which one should look. Conceivably, a perfect choice of parameters can be obtained by adjusting also the backward proportions $\overleftarrow{q}_{uv}$.

B Details about implementing Klee–Minty cubes

The natural way to associate a bit vector $(y_1, \ldots, y_n)$ to the vertices of a Klee–Minty cube (4) (Section 2.2) is derived from the standard cube: $y_i = 1$ if x_i is at the ceiling, i. e., $x_i = 2^i - 1 - x_{i-1}$, and $y_i = 0$ if x_i is at the floor, i. e., $x_i = x_{i-1}$. In this way, the Klee–Minty cube is mapped to the standard cube, and the n neighbors of $(y_1, \ldots, y_n)$ are obtained by simply flipping a single bit. The downside of this representation is that it is not easy to see whether an edge is improving.

Our program uses a different representation $(z_1, \ldots, z_n)$, which was introduced by Gärtner,

Klee–Minty cubes						
	$n = 10$			$n = 12$		
m	mean ± std. dev.	max.	$\hat{\chi}$	mean ± std. dev.	max.	$\hat{\chi}$
10^1	218.6 ± 226.5	7.47×10^2	0.34168	472.9 ± 661.9	2.03×10^3	0.43003
10^2	640.6 ± 1,830.8	1.30×10^4	0.20285	2,626.8 ± 11,882.5	8.69×10^4	0.33074
10^3	756.1 ± 5,094.5	1.33×10^5	0.17623	2,167.0 ± 27,495.6	8.44×10^5	0.38950
10^4	818.4 ± 9,017.6	5.12×10^5	0.06258	2,403.3 ± 36,794.5	2.26×10^6	0.09389
10^5	891.8 ± 20,241.5	5.13×10^6	0.05748	4,349.1 ± 5.99×10^5	1.88×10^8	0.43315
10^6	984.7 ± 31,787.0	1.26×10^7	0.01279	6,346.5 ± 2.53×10^6	2.35×10^9	0.37095
10^7	982.6 ± 50,113.4	6.09×10^7	0.00620	4,877.9 ± 3.17×10^6	9.30×10^9	0.19071
	1,024 ± 2.080×10^6			4,096 ± 6.025×10^{11}		
	#visited: avg = 37.8, max = 108			#visited: avg = 53.0, max = 148		

Table 9: Algorithm P for Klee–Minty cubes with Importance Sampling by the size of the reachable set

Klee–Minty cubes						
	$n = 10$			$n = 12$		
m	mean ± std. dev.	max.	$\hat{\chi}$	mean ± std. dev.	max.	$\hat{\chi}$
10^1	607.9 ± 396.9	1.28×10^3	0.21010	868.0 ± 1,456.5	4.92×10^3	0.56699
10^2	857.0 ± 1,552.7	1.27×10^4	0.14858	1,704.5 ± 4,347.4	4.04×10^4	0.23701
10^3	985.6 ± 3,287.0	5.65×10^4	0.05729	3,150.8 ± 13,846.9	2.47×10^5	0.07848
10^4	971.5 ± 3,667.1	1.16×10^5	0.01198	3,677.1 ± 37,719.8	3.33×10^6	0.09052
10^5	1,031.9 ± 8,148.2	1.93×10^6	0.01874	3,717.0 ± 52,576.6	1.26×10^7	0.03399
10^6	1,029.4 ± 11,760.9	9.25×10^6	0.00899	4,268.1 ± 2.40×10^5	2.18×10^8	0.05108
10^7	1,025.8 ± 9,644.3	1.21×10^7	0.00117	4,053.0 ± 1.22×10^5	2.18×10^8	0.00538
	1,024 ± 1.071×10^4			4,096 ± 9.419×10^5		
	#visited: avg = 42.5, max = 112			#visited: avg = 58.7, max = 152		

Table 10: Algorithm P for Klee–Minty cubes with Importance Sampling by the size of the reachable set divided by the indegree

Henk, and Ziegler [16]:

$$\begin{aligned} z_n &= y_n \\ z_{n-1} &= y_n \oplus y_{n-1} \\ z_{n-2} &= y_n \oplus y_{n-1} \oplus y_{n-2} \\ &\vdots \\ z_1 &= y_n \oplus y_{n-1} \oplus y_{n-2} \oplus \cdots \oplus y_2 \oplus y_1 \end{aligned}$$

It is the sequence of partial sums of the y_i's using addition modulo 2, or the exclusive-or operation $\oplus$.

The outgoing arcs are identified by the zero bits in this representation. If we follow such an arc, we must flip the i-th bit together with all lower-order bits. The reason is that while the orientation of all incident edges with higher index is maintained, the lower-index directions are perverted.

More concretely, we represent a vertex by the n-bit integer $(z_n z_{n-1} \dots z_2 z_1)_2$. It turns out that the value of this integer is equal to the x_n-coordinate of the vertex.

C Purdom's partial backtracking method

Purdom [31], in 1978, suggested the following variation of Knuth's Path Sampling algorithm for trees: Instead of proceeding to a single child, proceed to at most m randomly selected children. The parameter m can itself be chosen at random. In particular, Purdom looked at the setting of binary trees, and he proposed to choose $m = 2$ or $m = 1$ with probability p and $1-p$ respectively.

Purdom analyzed the number of visited nodes as well as the variance of the estimated tree size, for a certain class of random binary trees. He also made experiments with a backtracking program.

This method could be easily adapted to DAGs, but it will probably not compete with the more sophisticated Team Sampling methods, which allow a more direct control of the number of visited nodes. Still, one sentence from the conclusion of the paper is worth citing: "The analysis suggests that good results can be obtained by choosing the number of branches to investigate so that each level, from the root down to the bulk of the nodes, is examined at about the same frequency." This is in line with the strategy of the Crowd Sampling scheme, which sets a fixed budget B. In case of a tree, Crowd Sampling would indeed look at up to B nodes in each level.

D Index of important notation

$A, A', \hat{A}$ active set (list, or multiset) of nodes in Team Sampling (Section 5.1) and its variations
B budget: bound on the size of the active set in Crowd Sampling (Section 5.3)
$c(v), c(uv), c(G)$ node costs, arc costs, total cost of the graph (Section 1, Section 6.1)
$c_e(G)$ total cost with path weights, the expectation of X (Section 1.3, Theorem 3)
$\hat{\chi}$ Chatterjee–Diaconis score (Section 2.1)
$d^+(v), d^-(v)$ outdegree, indegree (Section 1)
$D_u(W)$ random value of all future additions to X when u is entered with weight W (Section 3)
$e_{uv}, e(P)$ arc weights, path weights (Section 1.3); assumed positive in Section 5
$E[\,]$ expectation
F_u random factor with $D_u(W) = F_u W$ (Section 3)
$f_v = \sum_w \mathcal{P}(v \to w)c(w) = E[F_v]$, coefficients in the invariant for Team Sampling (Section 5.4)
$G = (V, E)$ the input DAG with node set V and arc set E (Section 1)
$h\colon V \to U$ heuristic function, which assigns nodes to buckets (Section 5.2)
m number of repetitions in the experiments (Section 2.1)
μ_j mean of F_{v_j} for a successor node v_j (Section 3)
n dimension of the Klee–Minty cube (Section 2.2); number of permuted elements (Sections 2.1, 6.2.4)
$\mathcal{P}(v \to w)$ sum of the path weights $e(P)$ over all paths P from v to w (Section 5.4)
$P = u_0 u_1 \dots u_k$ path (Section 1.3)
$\Pr[\,]$ probability
$\vec{p}_{uv}$ forward proportions (Section 1.1); standard choice $\vec{p}_{uv} = 1/d^+(u)$
$\overleftarrow{q}_{uv}$ backward proportions (Section 1.1); standard choice $\overleftarrow{q}_{uv} = 1/d^-(v)$
R_v number of nodes reachable from v in a Klee–Minty cube (Section A.1)
s_0 source node (Section 1)
$S = \{u_1, \dots, u_k\}$ subset of A to be condensed in Team Sampling (Section 5.1, Section 5.5)
$S_1, \dots, S_i, \dots, S_B$ the sets that are condensed in Crowd Sampling (Section 5.3)
(U), (U$_q$), (U$_1$), (U$_e$) variations of the weight update step (Section 1)
U universe that indexes the buckets (Section 5.2)
var( ) variance (Section 3)
var$_j$ variance of F_{v_j} for a successor node v_j (Section 3)
W, W_v current weight (Section 1); the value of W when v is visited (Section 1.4)
$W[v]$ weight of an active node $v \in A$ in the Team Sampling algorithm (Section 5.1)
W_i, W_i' weights before and after condensation (Section 5.5)
X the current accumulated estimate (Section 1); X^{current}, X^{final} specific values of X (Section 5.4)
#visited, #generated, #saved: statistics in the tables (Sections 5.2–5.3)